Reto HTML + Tailwind — Nivel Alto

"un formulario es una conversación con el usuario;
si la conversación es clara, los datos llegan limpios"

Formulario de matrícula · Duración máxima: 60 minutos · Trabajo individual

Introducción.
En el reto del día, el equipo de *DataTech Solutions* concluyó que un sistema académico necesita el rigor de una base de datos relacional: tipos estrictos, integridad y datos consistentes. Pero la calidad de los datos empieza antes de la base de datos: empieza en el formulario. El equipo debe construir el formulario de matrícula de una institución educativa, como el de inscripción de una universidad, que valide la información y le muestre al aspirante sus errores mientras llena los campos, sin escribir una sola línea de JavaScript: solo validación nativa de HTML5 y variantes de Tailwind CSS.

Objetivo general.
Construir un formulario de matrícula accesible y responsive que valide los datos con atributos nativos de HTML5 y comunique visualmente cada estado (vacío, inválido, válido, seleccionado) usando únicamente variantes de Tailwind CSS.

Objetivos específicos.

- Elegir el atributo de validación nativa correcto para cada dato (`required`, `type`, `pattern`, `minlength`, `maxlength`, `min`, `max`).
- Agrupar campos con `fieldset` y `legend`, y asociar cada control con su `label` y su texto de ayuda mediante `aria-describedby`.
- Mostrar los errores solo después de que el usuario interactúe con el campo, usando `user-invalid:` y `peer`.
- Construir controles personalizados (tarjetas de selección) sobre inputs nativos usando `has-[:checked]:` y `group`.

Reglas del reto.

- Un único archivo `index.html`. Tailwind CSS v4 se carga solo con el script oficial del CDN (ver punto de partida); es el único `<script>` permitido.
- Prohibido: la etiqueta `<style>`, el atributo `style=""`, archivos `.css`, JavaScript propio (incluidos atributos como `onclick`) y librerías de componentes (DaisyUI, Flowbite, Preline, etc.).

- Imágenes: usa `https://picsum.photos/seed/<nombre>/600/600` o íconos SVG escritos en línea.
- Tiempo máximo: **60 minutos**. Al terminar el tiempo se entrega lo que funcione; es mejor una parte impecable que todo a medias.
- Los valores arbitrarios solo se permiten dentro de variantes como `has-[:checked]:`; no los uses para medidas que ya existen en la escala.

Punto de partida.
Copia esta estructura en tu `index.html` y construye todo dentro del `<body>`:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Matrícula | DataTech Academy</title>
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
</head>
<body class="bg-slate-50 text-slate-800 antialiased">
  <!-- Tu solución va aquí -->
</body>
</html>
```

Qué se espera de la solución.
Un formulario que no deje enviar datos incorrectos, que no «regañe» al aspirante antes de que escriba (los errores aparecen solo después de interactuar) y que se pueda completar entero con el teclado (Tab, Shift+Tab, flechas y Espacio). Además, el coder deberá responder en su README:

- ¿Por qué la validación del frontend nunca reemplaza las restricciones de la base de datos relacional (`NOT NULL`, `CHECK`, `UNIQUE`)?
- ¿Qué diferencia de experiencia notaste entre usar `invalid:` y `user-invalid:`?

El coder debe ejecutar y documentar los siguientes pasos:

1. **Estructura y accesibilidad (10 min):** un `form` con tres `fieldset` y su `legend`: *Datos personales* (nombre completo, tipo de documento con `select`, número de documento, fecha de nacimiento, correo y celular), *Programa académico* (programa, jornada y cursos electivos) y *Confirmación* (motivación y aceptación de términos). Cada control tiene `label` con `for`/`id` y un texto de ayuda enlazado con `aria-describedby`.

2. **Reglas de validación (10 min):** documento numérico de 6 a 10 dígitos (`pattern="[0-9]{6,10}"`), celular colombiano de 10 dígitos que empiece por 3, correo con `type="email"`, fecha de nacimiento con `min` y `max` que solo admitan aspirantes de 16 a 70 años, motivación de 50 a 300 caracteres y términos obligatorios. Deja al menos un campo opcional para contrastar.
3. **Estados visuales de validación (15 min):** borde rojo cuando el campo es inválido después de interactuar (`user-invalid:`), borde verde cuando es válido (`user-valid:`) y anillo de foco visible (`focus-visible:`). Cada campo tiene un mensaje de error oculto que aparece con `peer-user-invalid:block`.
4. **Tarjetas de selección (15 min):** la jornada (Mañana, Tarde, Noche) se elige con tarjetas grandes: el radio va dentro del `label` con `sr-only` y el label cambia borde y fondo con `has-[:checked]:`. Un ícono de check aparece solo en la tarjeta elegida (`group-has-[:checked]:visible`) y la tarjeta muestra el foco del teclado con `has-[:focus-visible]:ring-2`. Los electivos usan checkbox con `accent-*`.
5. **Layout y envío (10 min):** en `lg` el formulario ocupa 2 de 3 columnas y a la derecha va un `aside` «Resumen de matrícula» que se queda fijo al hacer scroll (`lg:sticky lg:top-6`). El botón Enviar se ve deshabilitado (opacidad baja y sin eventos) mientras exista algún campo inválido, usando `class="group"` en el form y `group-invalid:` en el botón.

Pistas técnicas (no son la solución).

- `user-invalid:` solo se activa después de que el usuario modifica el campo o intenta enviar; `invalid:` se activa desde que carga la página.
- `peer` solo afecta a hermanos posteriores: el `input` con `class="peer"` debe ir antes del `p` del mensaje y dentro del mismo padre.
- El elemento `form` también es `:invalid` si cualquiera de sus controles lo es; por eso funciona `group-invalid:`.
- `sr-only` oculta visualmente el radio pero lo mantiene accesible. Nunca uses `hidden` en un input que el usuario debe operar.
- Usa `autocomplete` (name, email, tel, bday) e `inputmode="numeric"` para mejorar la experiencia en el celular.

Reto extra (opcional, no suma tiempo).

- Un mensaje «Todo listo para enviar» que aparezca solo cuando el formulario es válido (`group-valid:block`).
- Ocultar el `aside` al imprimir (`print:hidden`) para que el formulario se vea limpio con Ctrl+P.

- Modo oscuro con la variante `dark:`.

Criterios de evaluación.

Criterio	Peso
Semántica y accesibilidad (labels, fieldset, aria, uso con teclado)	25 %
Reglas de validación correctas para cada dato	20 %
Estados visuales y mensajes de error	25 %
Tarjetas de selección y layout responsive	20 %
README y respuestas de reflexión	10 %

Entregables.

- Enlace del repositorio de la actividad en GitHub.
- Archivo `index.html` con la solución.
- Archivo README.md estructurado que contenga:
  - Título y nombre del coder.
  - Capturas del formulario con errores y del formulario válido, en móvil y escritorio.
  - Tabla con cada campo, su regla de validación y el atributo HTML usado.
  - Respuestas a las preguntas de reflexión.

Nombre Reto HTML + Tailwind — Nivel Alto	**Creado por** Jorge Areiza
Fecha de Creacion 21 de octubre del 2026, Medellin	**Ultima actualizacion**
Version 1.0.0	**Comentarios** Duración máxima: 60 minutos. Solo HTML + Tailwind CSS v4 (CDN). Sin JavaScript. Trabajo individual.